

ConTagion - Team14

ConTagion - Team14 (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): A 2D, introspective, story-based survival platformer set in the ruins of a city trapped 10,000 feet underground after an apocalyptic earthquake. The player navigates out with the help of a pet dog — the only one able to sense safe routes — avoiding unstable structures, falling debris, aftershocks, cannibalistic humans and infected creatures, across at least 5 levels with 4 different endings. Named technical features: gyro-based environment control for hazards/traps; player movement + pet command + collision; dog AI pathfinding + threat detection; enemy AI (idle/chase/attack) for player and dog. Custom tools: a scene/tile-map editor (falling structures, hazards like exposed wiring and fire), an enemy controller (spawn/detection/attack range), a player & dog controller, an obstacle trigger (aftershocks, cave-ins, enemies) and inventory/resource configuration. A server-side service stores inventory, level and completion time.

References:

- [Never Alone \(Kisima Injitchuna\) — Wikipedia](#) (atmospheric 2D puzzle-platformer; swap between a girl and her Arctic-fox companion to clear hazards)
- [Little Nightmares — Wikipedia](#) (2D survival-horror puzzle-platformer; a child evades grotesque/carnivorous enemies with only a lighter)
- [INSIDE — Wikipedia](#) (Playdead's dark 2D survival-platformer; evade pursuers and hazards, with a hidden alternate ending)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	ConTagion
External Database	✓
Authentication »	●
Analytics »	✓
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile + Web browser on mobile — the gyro/tilt hazard control and touch controls are mobile-first (the device gyroscope favours a native build), with a mobile-web front-end for reach; the Native PC build is the development/tools vehicle (device: Pixel 9a). **Extended:** Firebase + Linux server set up by M2 (full use T2).

Front end: Side-scrolling level renderer (player, dog, enemies, hazards, debris), lighting/atmosphere for the dark ruins, HUD (health/stamina, inventory, dog-command state), and menus/ending screens (OpenGL ES).
Languages: Web — HTML/JS/CSS + C++→WASM (Emscripten, SDL2/3,

OpenGL ES); Native — Kotlin Native Android UI + C++ via JNI (Android Studio); OpenGL ES in a GLSurfaceView where realtime 2D is needed (no SDL on native).

Back end: Engine core (fixed-timestep loop, entity system, platformer physics & collision, hazard/aftershock simulation, dog AI pathfinding + threat detection, enemy AI state machines, inventory/resource systems) + a save/progress service. **Languages:** C++.

Backend scope: Local now, server-ready (T2) — save/inventory/completion-time seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the fixed-timestep C++ loop is load-bearing — it steps player movement (run/jump/climb) and stamina, the dog's pathfinding and threat detection, enemy AI (idle/chase/attack) against both player and dog, hazard/aftershock triggers (falling debris, cave-ins, fire, flooding) and collision resolution, plus the gyro input that tilts/destabilises structures, all in real time.
- *Data-driven:* levels/tile maps, hazards and triggers, the two enemy types and their behaviour parameters, the dog's command set, survival resources/items and the 4 ending conditions are all defined as hot-reloadable JSON — no hard-coded levels, enemies or hazards.
- *2D:* side-scrolling sprite/tile rendering of the ruined city, player, dog, enemies, debris and hazard VFX, with atmospheric lighting (custom-rendered). *3D:* not needed. *Networking:* optional — cloud save/leaderboard only; no real-time netcode.
- *Custom tool:* a scene / tile-map / hazard editor (place tiles, falling structures, exposed wiring, fire, spawn points and aftershock/cave-in triggers; set player/dog spawns; configure enemy spawn/detection/attack ranges; author inventory/resources) with live preview.
- *External DB:* server-side inventory, level reached and completion time. *Auth* »: optional accounts. *Analytics* »: death/failure funnels — which hazard, enemy and level the run ends on, for balancing.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window and a fixed-timestep loop stepping player/dog/enemy updates, hazard triggers and collision, plus the gyro/tilt input that destabilises structures; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; M1G5 physics, M1G8 dog/enemy AI and the hazard system run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Entity/component registry in C++ with the player, dog, enemy types, hazards/triggers, items and level/tile data defined in JSON (nlohmann/json) with hot-reload.
- **Prerequisites:** G1 (loop & entity update hooks).
- **Feeds into:** M1G14 (tile/level data), M2G2 (the editor edits this data), M2G13 (inventory/progression reads it).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays for AI state, hazard triggers and collision, Tracy profiling, Catch2 stress tests spawning many enemies + simultaneous aftershock/debris hazards, GitHub Actions Windows+Linux matrix.
- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening) and the frame budget M2G1 validates for web/mobile.

- **G5 Physics & Collision** — run/jump/climb movement, falling debris, collapsing structures and hazard collision are core to the platforming.
Tech & dependencies
 - **Tech:** C++ + GLM math, custom integrator for platformer movement (run/jump/climb), broadphase (uniform grid / quadtree), collision resolution for the player, dog, debris and hazards.
 - **Prerequisites:** G1 (loop & tick).
 - **Feeds into:** M2G11 (advanced physics for cave-ins/falling structures); the dog and enemies that move through the world.
- **G8 Enemy AI & Pathfinding** — the dog's safe-route pathfinding + threat detection AND the two enemy types' idle/chase/attack behaviours.
Tech & dependencies
 - **Tech:** A* / flow fields for the dog's safe-route finding and enemy navigation, steering behaviours, and finite-state machines for idle/chase/attack plus the dog's follow/stay/scout and flee/cover states (custom C++).
 - **Prerequisites:** G1 (tick), G2 (enemy/dog data), G5 (movement/collision).
 - **Feeds into:** M2G10 (behavior trees, difficulty tiers); threat-alert feedback to the player.
- **G4 2D Graphics** — the side-scrolling render of the ruined city, player, dog, enemies, debris and hazards is the whole read of the game.
Tech & dependencies
 - **Tech: Web build:** Sprite/tile renderer with camera and batching for the level, player, dog, enemies and hazard VFX; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType HUD text. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** G1 (loop & window).
 - **Feeds into:** M1G10 (HUD over it), M2G4 (atmospheric lighting/post-FX for the dark ruins); every visible feature.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling of AI/hazard spikes, ASan on MSVC, memory pools for enemies/debris/particles, and Emscripten WASM validation for the mobile-web target plus on-device Pixel 9a checks (including gyro input); Linux server set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** All M1 goals (hardens the M1 codebase).
 - **Feeds into:** The web/mobile deliverable and headroom for dense hazard/enemy scenes.
- **G2 ★ Content Editor — Core** — the scene / tile-map / hazard editor: place tiles, falling structures, wiring/fire hazards, spawns and aftershock/cave-in triggers; set player/dog spawns; configure enemies; author inventory.

Tech & dependencies

- **Tech:** ImGui editor authoring tile maps, hazard/trigger placement, enemy spawn/detection/attack ranges, player/dog spawns and item/resource tables as JSON, with round-trip serialization and an undo/redo command stack.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (live level preview).
- **Prerequisites:** G2 (entity/level data), M1G14 (tile/level system).
- **Feeds into:** M2G6 (advanced tool features), M2G16 (balancing edits applied through it).

• G3 ★ CI — CMake, Code Quality, Build Standards & Packaging

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, and packaging of the native build plus the Emscripten web build artifact via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI and test foundation).
- **Feeds into:** Release/showcase builds of every ConTagion goal.

- **G10 AI — Advanced** — behavior trees and difficulty tiers for the dog and the two enemy types (idle/chase/attack, threat detection, flee/cover; timed puzzle pressure from the cannibals).

Tech & dependencies

- **Tech:** Behavior trees, squad/awareness logic and difficulty tiers over the M1 AI foundation (custom C++); the dog's scout/alert and the cannibals' timed-puzzle chase pressure.
 - **Prerequisites:** M1G8 (AI & pathfinding foundation).
 - **Feeds into:** Showcase encounters; M2G16 balancing.
- **G11 Physics — Advanced** — collapsing structures, cave-ins, falling debris and flooding as constraint/continuous-collision physics.

Tech & dependencies

- **Tech:** Constraint/joint solvers and continuous collision for collapsing structures, debris piles and flooding, GLM math, tuned from data.
 - **Prerequisites:** M1G5 (physics & collision).
 - **Feeds into:** The aftershock/cave-in set pieces; showcase gameplay depth.
- **G13 Gameplay Systems II** — progression across the 5 levels and 4 endings, inventory and the survival-resource economy (torchlight, food, items).

Tech & dependencies

- **Tech:** C++ systems over the entity/data model: level progression and branching to the 4 endings, inventory, resource pickups/consumption and the save/completion-time service.
- **Prerequisites:** G2 (entity system), M2G2 (editor authors the content).
- **Feeds into:** The 4-ending showcase; M2G16 telemetry balancing.

Track Goals & Team Composition

Team composition (set by instructor): 6 CS + 1 Design + 0 Art (7) — a story-driven survival platformer with a player, a dog companion, two enemy types, a ruined-city environment across 5 levels and heavy hazard/animation/VFX demand is a large character + environment + animation art surface that now has **no artist**, so the single designer owns the survival loop, level/puzzle design and the 4-ending narrative *and* takes the **D11 conditional** as a real graded Design goal each milestone (M1 Art Proof of Concept / M2 Art Implementation Quality — the last Design goal each milestone), which consumes one Design+UI slot per milestone (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). **Art-risk flag:** this is a very art- and animation-forward game (character roster + ruined-city environment + hazard/VFX + the 4 ending cutscenes), so 0 artists is a serious risk — scope levels/roster hard and lean on a reusable silhouette-first style under D11, or reconsider adding an artist within the $D+A \leq 3$ cap (a solo designer + 0 artists is the case the design pool flags, and squeezes the designer's systems/balance goals). *Scope note (advisory):* 5 levels × 4 endings × 2 enemy types is ambitious — treat 2–3 polished levels + the 4 ending hooks as the M2 showcase floor and let the editor scale content afterward.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the sense-route → navigate → evade → survive → escape loop and the 4 endings; benchmark e.g. INSIDE / Little Nightmares for readable dread and hazard pacing)
- M1D2 Core Gameplay Loop Prototype (greybox run/jump/climb + dog command/scout proving the companion-guided survival loop is tense and fun) · prototype in Unity
- M1D11 Art Proof of Concept *(conditional — no artist; Design-owned)* — the visual direction: the player + dog companion, the two enemy types, the ruined-city environment and the hazard/VFX look; 2 mockups. *(Replaces the M1D6 economy-design slot the artist's absence consumes.)*

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness

- M2D11 Art Implementation Quality *(conditional — no artist; Design-owned)* — the player + dog, the two enemies, the ruined-city levels, hazards and VFX implemented in-engine, no placeholders.
- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — screen-flow map and readability standards for the HUD (health/stamina, dog-command state, inventory) and the threat-alert cues.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — standard screens + the dog-command and inventory/interaction flows navigable unaided.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished HUD and menus, touch + gyro affordances confirmed on the Pixel 9a / mobile-web target.
- **Art:** 0 artists — the large art/animation surface (the player + dog + 2 enemies, the ruined-city environment, hazards/VFX and the 4 ending cutscenes) is covered by the D11 conditionals, taken as Design goals by the single designer (see Design M1/M2). This is a very art-forward game, so 0 artists is a serious risk — scope levels/roster hard and lean on a reusable silhouette-first style.
- **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (oppressive underground ambience, footsteps/dog, aftershock rumble, enemy stings), suitable and correctly triggered.
 - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback (overlapping hazard/enemy/ambience SFX).
- **TEAM M2:**
 - M2TA1 ★ Overall Audio Experience — standard Games floor (atmospheric BGM + hazard, enemy, dog and UI SFX, ≥6 types), tension-reactive where the design calls for it.
 - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; spatial cues for off-screen threats the dog senses).

- M2TP1 ★ Final Presentation — 7 min in front of class +
instructors: live custom-tools showcase, prototype demo, team
post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*